

VITyarthi - Build Your Own Project

Image Processing Basics with OpenCV

Project Report

Course concept: Fundamentals of image processing using OpenCV and Python in Jupyter Notebook

Tools: Python, OpenCV, NumPy, Matplotlib, Jupyter Notebook, Pytest

1. Introduction

Image processing is the process of applying operations to digital images so that they can be viewed, transformed or analysed more effectively. This project was built around the introductory OpenCV operations covered in the VITYarthi session. Instead of keeping each command as an isolated notebook example, the project combines them into a small reusable workflow that accepts an image and automatically creates several processed versions.

2. Problem Statement

When beginners first work with OpenCV, operations such as imread, imwrite, grayscale conversion, channel splitting and blurring are usually demonstrated separately. The project addresses this by putting these operations into one structured pipeline with validation, reusable functions, saved outputs and a notebook demonstration.

3. Objectives

The main objectives are to read and validate an input image, save image outputs, convert an image to grayscale, separate BGR colour information, apply basic blur filters, organise the code into modules, and provide a notebook and automated tests that make the results easy to verify.

4. Functional Requirements

- FR1 - The system shall accept the path of an input image.
- FR2 - The system shall validate that the file exists and can be read by OpenCV.
- FR3 - The system shall convert the image to grayscale.
- FR4 - The system shall split and visualise Blue, Green and Red channels.
- FR5 - The system shall apply average blur and Gaussian blur.
- FR6 - The system shall save all generated results into an output directory.
- FR7 - The system shall create a text summary of the processed image.

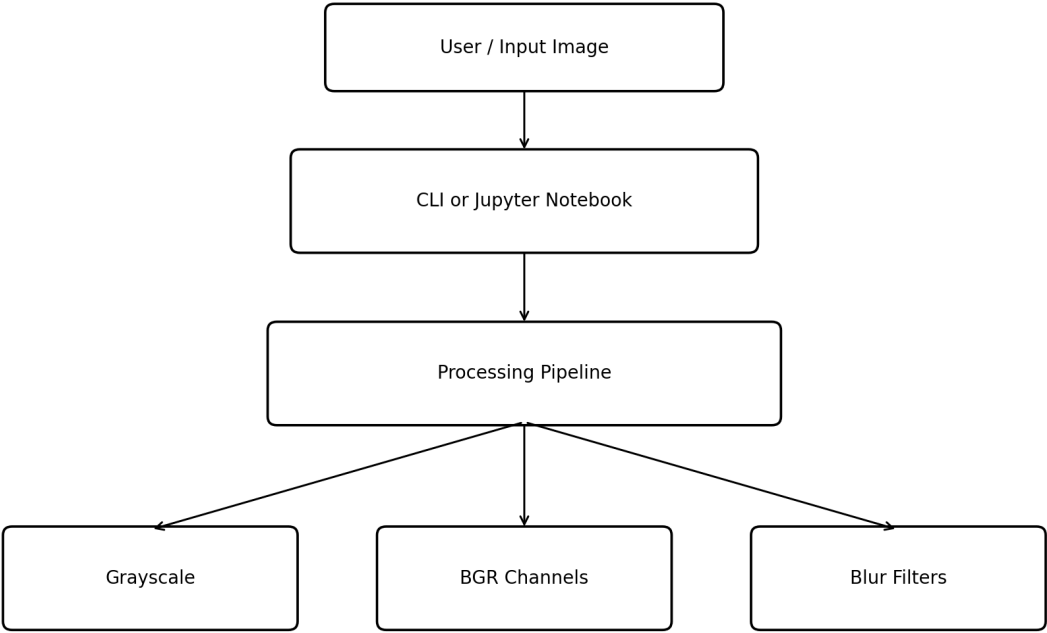
5. Non-functional Requirements

Requirement	How it is addressed
Usability	Single command execution and a notebook with step-by-step cells.
Reliability	Invalid image paths and unreadable files produce clear errors.
Maintainability	Processing operations are separated into small Python modules.
Performance	Only basic OpenCV array operations are used; no heavy model is loaded.
Resource efficiency	Processing is local and uses only the input image and generated files.

6. System Architecture

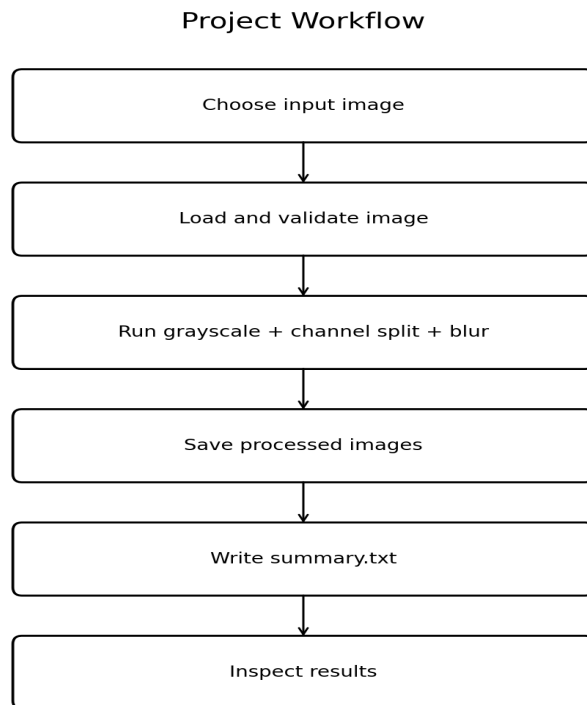
The user can run the project through the command line or study the same processing steps in the Jupyter notebook. The processing pipeline calls separate modules for image I/O, grayscale conversion, BGR channel operations, blur filters and result summary generation.

System Architecture

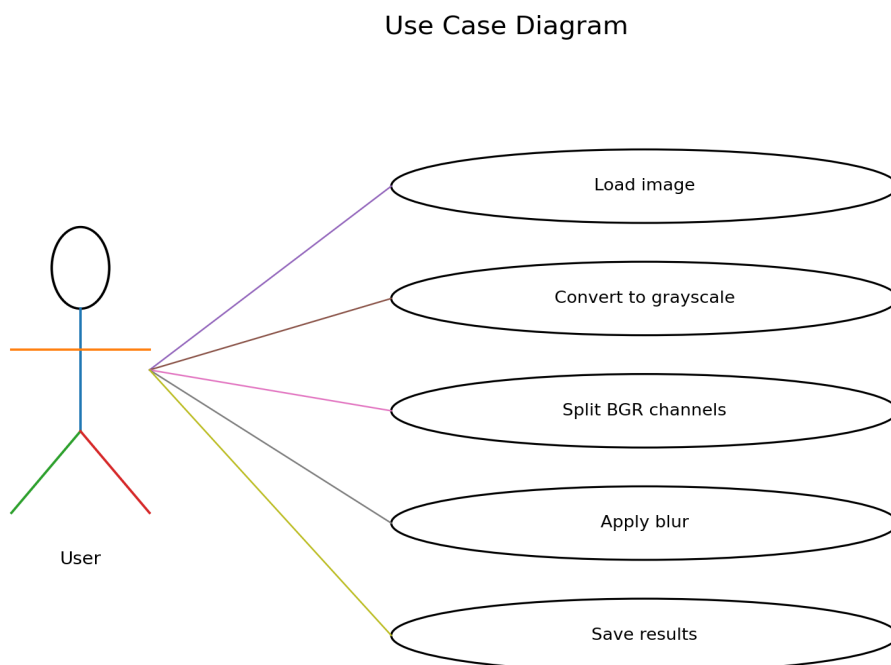


7. Design Diagrams

Workflow Diagram

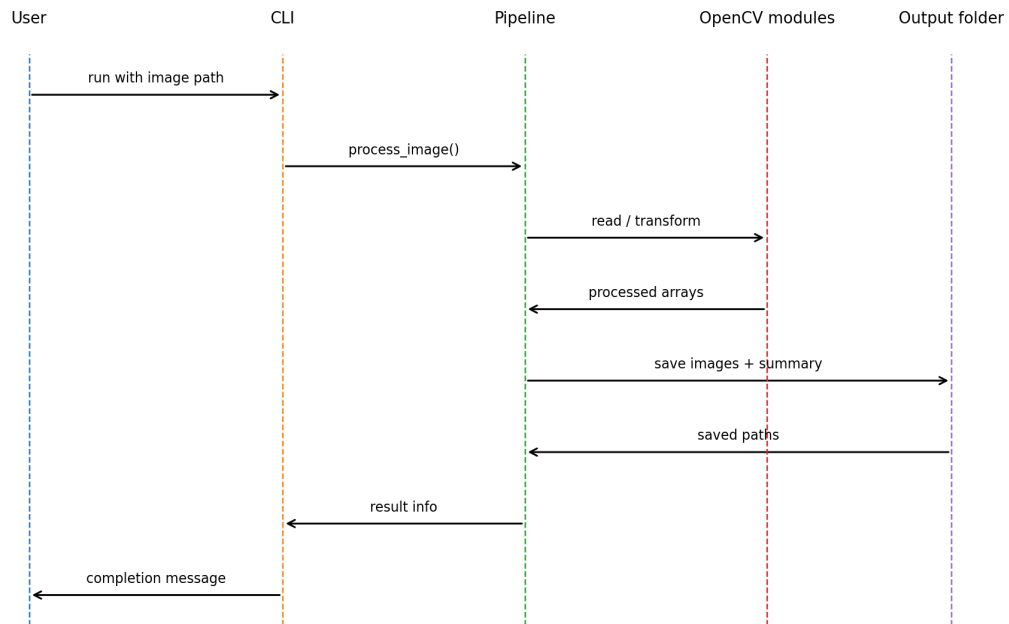


Use Case Diagram



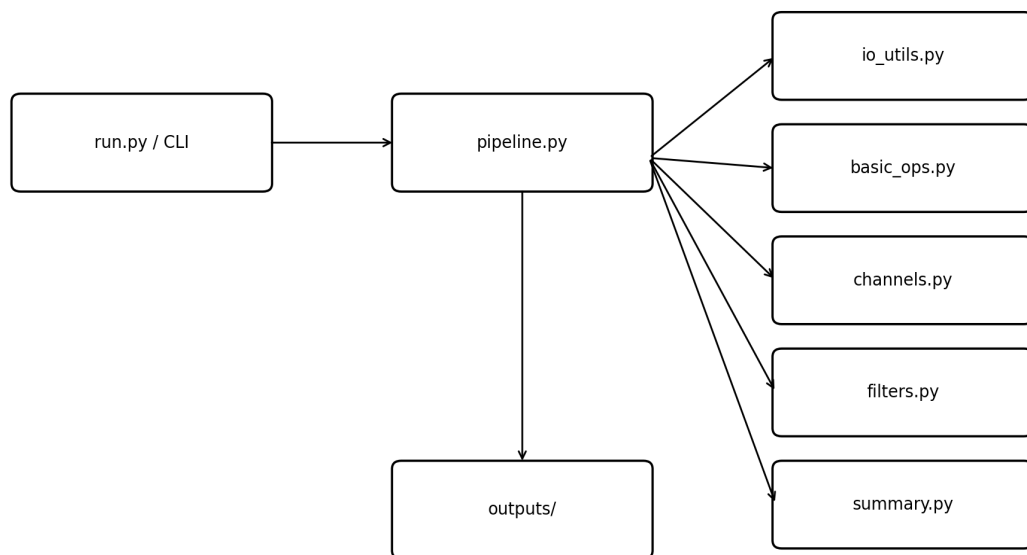
Sequence Diagram

Sequence Diagram



Component Diagram

Component Diagram



8. Design Decisions and Rationale

Modular files: Each concept is placed in a small module so that it is easy to connect the code to the topic learned.

Matplotlib in notebook: `cv2.imshow()` opens a separate GUI window and can be inconvenient in notebook environments, so Matplotlib is used for notebook display while OpenCV performs the image processing.

BGR-aware channel handling: OpenCV uses BGR channel order, so the project keeps this convention in processing and converts to RGB only when required for Matplotlib display.

Command-line interface: A CLI makes the project reusable with different images without changing the source code.

Plain output files: JPG output files and a text summary are easy to inspect and do not require a database.

9. Implementation Details

File	Responsibility
io_utils.py	Load, validate and save images.
basic_ops.py	Grayscale conversion.
channels.py	BGR splitting and channel visualisation.
filters.py	Average blur and Gaussian blur.
summary.py	Image metadata and text report.
pipeline.py	Coordinates the full processing workflow.
cli.py	Command-line arguments and user-facing execution.
image_processing_demo.ipynb	Notebook demonstration of the same concepts.

10. Results

Running the program on the included sample image produces an original copy, a grayscale image, three colour-channel visualisations, an average-blurred image, a Gaussian-blurred image and a summary text file.

The output folder is regenerated each time the user runs the project. The included sample image allows the repository to be tested immediately after installation.

11. Testing Approach

Pytest is used for lightweight validation. Tests check the shape of grayscale output, correctness of channel separation, preservation of image dimensions after blurring, rejection of invalid kernel sizes and successful generation of all expected files by the complete pipeline.

12. Challenges Faced

Two practical issues are important in this project. First, OpenCV uses BGR channel ordering while Matplotlib uses RGB, which can make displayed colours look incorrect if no conversion is performed. Second, blur kernels used by GaussianBlur must use valid positive odd dimensions, so the project validates the kernel before processing.

13. Learnings and Key Takeaways

- How OpenCV represents an image as a NumPy array.
- How imread and imwrite are used for image input and output.
- Why grayscale images have one channel while colour images normally have three.
- How B, G and R channels contribute to the appearance of a colour image.
- How neighbourhood-based filters create a blur effect.
- Why validation and small reusable functions make even a simple project easier to test.

14. Future Enhancements

- Edge detection using Canny.
- Image thresholding.
- Brightness and contrast controls.
- Histogram plotting.
- Image rotation and resizing.
- A simple graphical or web interface.

15. References

- OpenCV-Python documentation - image I/O and image processing functions.
- VITyarthi Session 5: Image Processing with OpenCV and Python - Hands On Session.
- NumPy documentation - array representation and operations.
- Matplotlib documentation - image display in Jupyter Notebook.